

Bijlage – Node-RED routing/decoding + msgType byte strategie (RAK3172 project)

Versie: 2026-01-03 • Doel: een zelfstandige bijlage bij de bring-up gids. Deze bijlage beschrijft hoe je binnenkomende LoRa payloads op je VPS/Node-RED kunt herkennen, routeren en decoderen zodat zowel legacy ASCII ("o2:temp:rh:bat") als nieuwe binaire 8B berichten naast elkaar kunnen blijven werken, zonder aanpassingen aan bestaande MKR1310 nodes in het veld.

1. Waarom we een msgType/version byte gebruiken (byte 0)

Je huidige backend (MQTT → Node-RED → Influx → Modbus/web) is operationeel en wordt door veel sensoren wereldwijd gebruikt. Daarom willen we zo min mogelijk veranderen aan de backend en vooral geen verplicht fPort-schema afdwingen (want veld-devices sturen niet allemaal identiek).

De eerste byte (byte 0) als msgType/version is de meest robuuste oplossing omdat:

- Hij werkt onafhankelijk van fPort en applicationName.
- Node-RED kan altijd eerst de payload-lengte controleren (bijv. 8 bytes) en daarna de msgType lezen.
- Je kunt meerdere sensor-types ondersteunen (O2, CO2, PH3, GPS, etc.) zonder de hele flow per type te dupliceren.
- Je kunt versiebeheer doen: msgType 0x01 = SensorPayload v1; later 0x11 = SensorPayload v2, etc.

1.1 Waarom niet alleen op lengte selecteren?

Alleen lengte is kwetsbaar: andere payloads kunnen toevallig dezelfde lengte hebben. Door lengte + msgType te combineren voorkom je dat je een payload verkeerd decodeert (zoals je al zag met een 'Onbekend msgType ...' melding).

2. Payload contract (8 bytes – SensorPayload v1)

Offset	Naam	Type	Schaal	Opmerking
0	msgType	uint8	—	0x01 = SensorPayload v1
1–2	o2_x10	uint16 LE	waarde/10	O2% = o2_x10 / 10
3–4	t_x10	int16 LE	waarde/10	Temp °C = t_x10 / 10
5–6	h_x10	uint16 LE	waarde/10	RH% = h_x10 / 10
7	bat_pct	uint8	0..100	Battery percentage

3. Aanpak in Node-RED: routeren op (len, msgType) en fallback naar legacy ASCII

Je wilt bij binnenkomst herkennen of het een nieuw binair bericht is of een oud ASCII bericht, en dan de juiste decoder aanroepen. Onderstaande aanpak is 'drop-in': je plaatst eerst een Router Function vóór je bestaande 'decode base64 + normalize' node.

3.1 Router Function (3 outputs: GPS | sensors | other) – uitgebreid met binaire detectie

```
// Router v2: herkent GPS (10B), legacy ASCII (o2:temp:rh:bat) én binaire SensorPayload v1 (8B, msgType=0x01)
// Out1 = GPS      | Out2 = sensors (genormaliseerd) | Out3 = other
```

```
function toObj(m) {
  let p = m.payload;
  if (Buffer.isBuffer(p)) p = p.toString('utf8');
  if (typeof p === 'string') { try { p = JSON.parse(p); } catch {} }
  return (p && typeof p === 'object') ? p : {};
}
```

```

const p = toObj(msg);
const topic = String(msg.topic || '');

// app/device uit topic (TTN-stijl)
let appFromTopic = '';
let devFromTopic = '';
if (topic) {
  const parts = topic.split('/');
  const aIdx = parts.indexOf('application');
  const dIdx = parts.indexOf('device');
  if (aIdx >= 0 && parts[aIdx + 1]) appFromTopic = parts[aIdx + 1];
  if (dIdx >= 0 && parts[dIdx + 1]) devFromTopic = parts[dIdx + 1];
}

const appName = String(
  p.applicationName ??
  p.end_device_ids?.application_ids?.application_id ??
  ''
).trim();
const appEff = appName || appFromTopic;

// fPort + base64 uit varianten
const fport = Number(p.fPort ?? p.uplink_message?.f_port ?? -1);
const b64 = p.data ?? p.object?.data ?? p.uplink_message?.frm_payload ?? p.frm_payload;
if (typeof b64 !== 'string' || b64.length === 0) return [null, null, msg];

let buf;
try { buf = Buffer.from(b64, 'base64'); } catch { return [null, null, msg]; }
const len = buf.length;

// === GPS detectie (zoals je al had) ===
const looksGPS =
  (fport === 2 && len === 10) ||
  (typeof appEff === 'string' && appEff.startsWith('GPS_')) ||
  (typeof appFromTopic === 'string' && appFromTopic.startsWith('GPS_'));

if (looksGPS) {
  node.status({ fill: 'green', shape: 'dot', text: `GPS fPort:${fport} len:${len} app:${appEff||'-'} ` });
  msg._route = 'gps';
  return [msg, null, null];
}

// === Nieuwe binaire sensors (8B) ===
if (len === 8) {
  const msgType = buf.readUInt8(0);
  if (msgType === 0x01) {
    node.status({ fill: 'blue', shape: 'dot', text: `BIN v1 len:8 app:${appEff||'-'} ` });
    msg._route = 'bin_sensor_v1';
    return [null, msg, null]; // naar sensor decode node (binaire decoder)
  } else {
    // 8 bytes maar onbekend type: stuur naar OTHER (of log)
    node.warn(`Onbekend msgType ${msgType} bij 8B payload`);
    msg._route = 'other';
    return [null, null, msg];
  }
}

// === Legacy ASCII sensors ===
let ascii = '';
try { ascii = buf.toString('utf8').trim(); } catch {}
let isLegacy = false;
if (ascii.includes(':') || ascii.includes(',')) {
  const sep = ascii.includes(':') ? ':' : ',';
  const parts = ascii.split(sep).map(s => s.trim());
  if (parts.length === 4 && parts.every(v => v !== '' && !Number.isNaN(Number(v)))) isLegacy = true;
}
if (isLegacy) {
  node.status({ fill: 'blue', shape: 'dot', text: `ASCII len:${len} app:${appEff||'-'} ` });
  msg._route = 'ascii_sensor';
  return [null, msg, null];
}

```

```
// === Other ===
node.status({ fill: 'yellow', shape: 'ring', text: `OTHER fPort:${fport} len:${len} app:${appEff}|-`});
msg._route = 'other';
return [null, null, msg];
```

4. Binaire decoder node (8B → measurements)

Plaats na de router (output 'sensors') een aparte Function node die alleen binaire v1 decodeert. Daarna geef je dezelfde msg.measurements structuur terug als je huidige flow, zodat je bestaande Influx/mapping nodes kunnen blijven staan.

```
// Decode BIN SensorPayload v1 (8 bytes) naar dezelfde normalize structuur als legacy flow.
// Verwacht: msg.payload = TTN/ChirpStack JSON (object of string), met base64 in data/frm_payload etc.

function asObj(x){ if (typeof x === 'string') { try { return JSON.parse(x); } catch(_){} } return x || {} }
const p = asObj(msg.payload);

// topic: application/<app>/device/<devEUI>/rx
const parts = (msg.topic||'').split("/");
const appIdx = parts.indexOf('application');
const devIdx = parts.indexOf('device');
const app = (appIdx>=0 && parts[appIdx+1]) ? parts[appIdx+1] : (p.applicationName || 'unknown_app');
const devTop = (devIdx>=0 && parts[devIdx+1]) ? parts[devIdx+1] : 'unknown_dev';

const device = p.deviceName || p.devEUI || p.end_device_ids?.dev_eui || devTop;

const b64 = p.data || p.object?.data || p.uplink_message?.frm_payload || p.frm_payload;
if (!b64) { node.warn('Geen base64 payload'); return null; }

const buf = Buffer.from(b64, 'base64');
if (buf.length !== 8) { node.warn(`Onverwachte lengte: ${buf.length}`); return null; }

const msgType = buf.readUInt8(0);
if (msgType !== 0x01) { node.warn(`Onbekend msgType ${msgType} bij 8B payload`); return null; }

// Little-endian decode
const o2_x10 = buf.readUInt16LE(1);
const t_x10 = buf.readInt16LE(3);
const h_x10 = buf.readUInt16LE(5);
const bat = buf.readUInt8(7);

const o2 = o2_x10 / 10.0;
const temp = t_x10 / 10.0;
const rh = h_x10 / 10.0;

// timestamp
let ts = Date.now();
const tsRaw = p.ts || p.uplink_message?.received_at || p.rxInfo?.[0]?.time;
if (tsRaw) {
  const t = Date.parse(tsRaw);
  if (!isNaN(t)) ts = t;
}

msg.app = String(app);
msg.device = String(device);
msg.devEUI = String(p.devEUI || p.end_device_ids?.dev_eui || device);
msg.measurements = { o2, temp, rh, bat };
msg.ts = ts;

// payload voor debug
msg.payload = { app: msg.app, device: msg.device, o2, temp, rh, bat, ts, fCnt:p.fCnt, fPort:(p.fPort ?? 0) };
return msg;
```

5. Legacy ASCII decoder – laten staan (ongewijzigd)

Je bestaande 'decode base64 + normalize' node voor ASCII kun je laten staan voor oude MKR-devices. Het enige wat verandert is dat je router nu selecteert welke decoder moet draaien.

6. Voorstel: doorontwikkeling van het msgType schema

Zodra je meerdere sensortypes wil ondersteunen (CO2, PH3, O2, etc.), kun je het msgType-byte uitbreiden. Hier is een praktisch voorstel dat backward compatible blijft:

msgType	Betekenis	Lengte	Decoder
0x01	SensorPayload v1 (O2/T/RH/BAT)	8	BIN v1 decoder (LE)
0x02	SensorPayload v1 (CO2/T/RH/BAT)	10 (bijv.)	CO2 decoder
0x03	SensorPayload v1 (PH3/T/RH/BAT)	8/10	PH3 decoder
0x10	GPS v1 (compact)	10	GPS decoder (bestaat al)
0x11	SensorPayload v2 (nieuwe layout)	variabel	v2 decoder

6.1 Aanbevolen evolutie (zonder grote backend wijziging)

- Houd byte0 = msgType altijd verplicht. Verander dit nooit; zo blijft routing stabiel.
- Leg per msgType een vaste payloadlengte vast (of gebruik TLV in v2).
- Gebruik little-endian consequent (Node-RED Buffer.read*LE).
- Reserveer 1 byte voor flags of een simpele checksum in een toekomstige v2 (optioneel).
- Voeg nieuwe decoders toe als losse Function nodes, maar laat de Influx/mapping node generiek op msg.measurements werken.
- Log 'onbekend msgType' naar een aparte debug/alert flow zodat je nieuwe sensortypes snel ziet.

7. Minimal-change integratie in jouw bestaande flow

Concreet:

- 1 Plaats Router v2 vóór de huidige ASCII decoder.
- 2 Maak twee decoder nodes: (A) BIN v1 decoder en (B) jouw bestaande ASCII decoder.
- 3 Laat daarna beide decoders doorlopen naar dezelfde '→ Influx points' node (msg.measurements blijft hetzelfde).
- 4 GPS-pad blijft zoals hij is.
- 5 Other-pad kan naar debug/log of future handlers.